

AI Usage Description

1. Overview of AI use

AI tools such as GPT-style and Claude-style assistants were used as support tools, mainly for:

- Translating selected ideas between Java/TypeScript and C++ syntax.
- Explaining C++/CMake/raylib issues during implementation.
- Drafting and reorganizing documentation text.
- Finding wording for the AI-usage, summary, and user-manual documents.

AI was not used as a runtime component. The submitted program does not call any AI service while running.

2. Files and modules where AI was used

AI assistance was most useful around documentation and language translation:

- Documentation drafts under `docs/`.
- CMake/build troubleshooting notes.
- Mechanical translation help when comparing prior Java chess code with the new C++ class layout.
- Small explanation or refactoring suggestions around C++ headers and function boundaries.

The final code was manually reviewed and adapted before being kept in the project.

3. How AI output was reviewed

AI output was treated as a draft, not as authoritative code. We checked it by:

1. Reading the generated code or text manually.
2. Comparing chess logic against known rules and prior chess programming experience.
3. Running unit tests, perft tests, and numerical reference tests.
4. Checking the tutorial recording and live raylib behavior manually.
5. Rewriting or deleting suggestions that did not fit the C++ architecture.

4. Where AI was wrong (and what we corrected)

AI tools were weak at several important project parts:

- Visual design: suggested raylib layouts were often cluttered, badly spaced, or not representative of neural-network activations. We redesigned the board, controls, and panels manually.
- Chess-rule details: generated explanations sometimes missed edge cases around castling, en-passant, check validation, and undo state. These areas were verified through tests and manual review.
- Neural-network shape details: model dimensions, policy sizes, and activation shapes had to be checked against the actual C++ implementation.
- Over-generalized documentation: AI drafts tended to say “AI generated documentation” too broadly. The final report states the real limited use: translation, explanation, structuring, and review support.

5. Hand-written modules

The core grading points were implemented and reviewed by the students:

- C++ class structure across `src/chess`, `src/game`, `src/nn`, `src/io`, and `src/viz`.
- Legal chess move generation, FEN/SAN handling, make/unmake, and perft tests.
- The manual linked-list move history.
- File I/O for config, FEN, PGN support, binary weights, and test data.
- Neural-network operations and activation snapshots.
- Raylib board rendering, controls, editor, activation panels, and tutorial demo.
- Build/test scripts and final project documentation.

6. Lessons learned

AI is useful for translation and documentation scaffolding, but it cannot replace domain understanding for a chess engine/visualizer. The project needed manual knowledge of chess rules, C++ ownership, raylib rendering, neural-network tensor shapes, and test design. The best workflow was to use AI for drafts and explanations, then verify every important result against code, tests, and live behavior.